

Ìtàn

*An offline agricultural extension agent for
smallholder farmers — no internet, no data cost,
no guesswork.*

Challenge	African Deep Tech Challenge (ADTC) 2026
Domain	Agriculture — on-device language model inference
Target	8 GB RAM · integrated graphics · x86 laptop · Ubuntu 22.04
Model	Qwen2.5-1.5B-Instruct · LoRA · Q4_K_M · llama.cpp
Projected	≈ 82-88 / 100 + 10 African use-case bonus
Licence	Apache 2.0 — open source, public repository
Status	Final — this document is the build plan, not a pitch

Ìtàn (Yorùbá: *story, account*). The merged blueprint of two independently written proposals — *Ìtàn* and *Àgbẹ̀* — that arrived at the same architecture. This document supersedes both. We are no longer proposing. We are building exactly what is written here.

- 1 The Challenge, and the Arithmetic That Decides It**
 - 1.1 What ADTC asks · 1.2 The fixed machine · 1.3 The scoring formula · 1.4 The 34-point thesis · 1.5 Two questions for the organisers
- 2 What We Are Building**
 - 2.1 One sentence · 2.2 The user · 2.3 Why cloud AI fails here · 2.4 Three pillars · 2.5 Feature set
- 3 System Architecture**
 - 3.1 Layered view · 3.2 The journey of a question · 3.3 The four-tier taxonomy · 3.4 Design decisions
- 4 The Knowledge Layer**
 - 4.1 Sources · 4.2 Ingestion pipeline · 4.3 The structured knowledge base
- 5 The Agent**
 - 5.1 Tools · 5.2 The bounded loop · 5.3 Guided pest triage · 5.4 The refusal policy
- 6 The Language Layer**
- 7 Fine-Tuning Plan**
- 8 Performance Engineering**
 - 8.1 Roofline · 8.2 Lookup decoding + phrase bank · 8.3 Fallback · 8.4 Budgets · 8.5 Thermal
- 9 Evaluation & Benchmarking**
 - 9.1 The harness · 9.2 Ablations · 9.3 Profiler compatibility
- 10 Build Plan — Eight Weeks**
 - 10.1 Sprints · 10.2 Milestones · 10.3 Critical path · 10.4 The cut order
- 11 Team, Ownership & Working Rules**
- 12 Risk Register**
- 13 Deliverables**
- 14 Why This Wins**

1 The Challenge, and the Arithmetic That Decides It

1.1 What ADTC asks

The African Deep Tech Challenge asks entrants to build a useful AI assistant that runs **entirely on an ordinary, inexpensive laptop** — no cloud, no internet, no GPU. The framing is deliberate: the barrier to AI across much of Africa is not that models are unavailable, it is that *access* to them costs data, connectivity and money many users do not have. We are entering the **Agriculture** track. Section 1.4 shows why that choice is strategic, not sentimental.

1.2 The fixed machine

Component	Reference specification
Memory	8 GB RAM total; a 7 GB budget appears in the scoring formula
Processor	Intel i5 10th–12th gen / Ryzen 5 3000–5000; laptop-class, thermally limited
Graphics	Integrated only — all inference on CPU
Storage / OS	256 GB SSD · Ubuntu 22.04
Network	None at evaluation time
Training budget	5 hours of GPU credit (Udutech) — enough for LoRA, nothing more

1.3 The scoring formula

```
S_total = 0.50 · S_acc + 0.30 · S_perf + 0.20 · S_eff - P_thermal

S_perf  = 100 × (TPS_actual / TPS_max), TPS_REFERENCE = 15.0
S_eff   = 100 × ((7 GB - Peak_RAM) / 7 GB)
P_thermal = -10 if throttled or T > 85°C, else 0
African use-case bonus: up to +10
```

The formula is published in advance. That converts this competition from an open-ended build into a **constrained optimisation problem**, and everything in this blueprint follows from taking that literally.

1.4 The 34-point thesis

Most competent teams will read “accuracy is weighted 50%” and reach for the largest model that fits in 8 GB — a 7B at Q4. This is the single most expensive mistake available in the competition, and we expect much of the field to make it. Model size does not only affect accuracy; **it dominates the other 50% of the score**:

Metric	7B @ Q4_K_M	1.5B @ Q4_K_M	Delta
Peak RAM (model + runtime)	≈ 4.9 GB	≈ 1.5 GB	−3.4 GB
S_eff weighted contribution (20%)	6.0 pts	15.7 pts	+9.7 pts
Tokens/sec on i5 + iGPU	≈ 5–7	≈ 26–34	≈ 5× faster
S_perf weighted contribution (30%)	5.4 pts	30.0 pts	+24.6 pts
Thermal risk under sustained load	High	Low	10 pts protected
Combined non-accuracy total	11.4 / 50	45.7 / 50	+34.3 pts

THE THESIS. Going small is worth roughly **34 points before a single accuracy question is asked**. A 7B model would need to beat a 1.5B by 68 raw accuracy points just to break even — impossible on a retrieval-grounded, domain-narrow task, where a well-built 1.5B RAG system typically lands within 5–12 raw points of a 7B and can exceed it, because retrieval supplies facts the 7B would hallucinate. Every design decision below follows from this one table.

If the model must be small, it cannot be asked to memorise. So: **knowledge moves to retrieval** (the model reads the urea rate from a retrieved FAO passage; it does not recall it), **arithmetic moves to tools** (every calculation runs as deterministic Python, converting a class of guaranteed failures into guaranteed correctness), and **the model's only job is reasoning and language** — decompose, select a tool, read the evidence, answer in the user's language with a citation. That is comfortably within 1.5B capability when fine-tuned for exactly that behaviour.

1.5 Two questions for the organisers — Day 1, in writing

Two rubric ambiguities change architecture decisions, so we resolve them before we build:

1. **Does “Peak_RAM” mean process RSS or the full process tree / system delta?** Worth ~5 points and it decides how much the UI framework matters (Section 8.4).
2. **Is S_perf normalised against the fixed TPS_REFERENCE of 15, or the maximum observed entry?** The published material is contradictory. If it is the observed maximum, the metric is a race; if fixed, we can already saturate it.

2.1 One sentence

Ìtàn is a fully offline advisory agent that answers real farming questions — pest identification, fertiliser dosing, planting calendars, spray dilution, margins — in English, Hausa, Yorùbá and Nigerian Pidgin, with a citation on every fact, on a commodity Ubuntu laptop with zero network dependency.

It is not a chatbot with a model bolted on. It is a retrieval-grounded, tool-calling agent in which a deliberately small language model acts as a reasoning and language layer, while all factual knowledge lives in a curated corpus of African agricultural extension literature and all arithmetic executes as deterministic Python.

2.2 The user

The ADTC brief names an extension officer in Arusha as an archetypal user. Ìtàn is built for that person and the farmers they serve. Sub-Saharan Africa runs roughly one extension officer per 1,000–3,000 farming households against an FAO-recommended ratio near 1:400. Where advice does not reach farmers, the failures are concrete and expensive: misidentified pests treated with the wrong pesticide, fertiliser at the wrong rate or growth stage, planting windows missed by weeks. Ìtàn puts a verified, citable knowledge base plus a language interface onto a machine that already exists in a local-government agriculture office, a cooperative, or an agro-dealer's shop.

2.3 Why cloud AI fails here

Barrier	Reality on the ground
Connectivity	Rural coverage is intermittent; advisory need peaks in the field, not in town
Data cost	Per-query mobile data is a real recurring cost against thin margins
API fees	Any per-token cost fails for a user earning seasonally
Latency	Round-trips over congested 2G/3G are unusable conversationally
Language	Frontier models are weak in Hausa, Yorùbá and agricultural Pidgin register
Knowledge	A general model does not know the maize window in Oyo differs from Kano — it answers anyway, fluently and wrongly
Trust & privacy	Farm data, yields and finances leaving the device is a genuine concern

Every barrier is eliminated by on-device inference. That is the thesis of the entire challenge, and agriculture is where it bites hardest.

2.4 The three pillars

Pillar 1 — Retrieval, not memorisation. ~10,000 curated chunks from FAO, IITA, CGIAR, CABI PlantWise and national extension handbooks. Retrieval is hybrid — dense embeddings plus BM25 — because agricultural queries contain rare proper nouns (cultivars, active ingredients, pest binomials) that dense retrieval alone handles poorly.

Pillar 2 — Tools, not arithmetic. Fertiliser dosing, seed rate, spacing, spray dilution and margins execute as Python functions, not generated tokens. The single highest-leverage accuracy intervention available at 1.5B scale.

Pillar 3 — Language, not translation. A term-normalisation layer maps local vocabulary to corpus terminology before retrieval runs (“gero” retrieves pearl millet; “ọkà” retrieves maize). The model then responds in the query's language, with the LoRA trained specifically on code-switched agricultural instruction data.

2.5 Feature set

Core — must ship (frozen after Week 4)

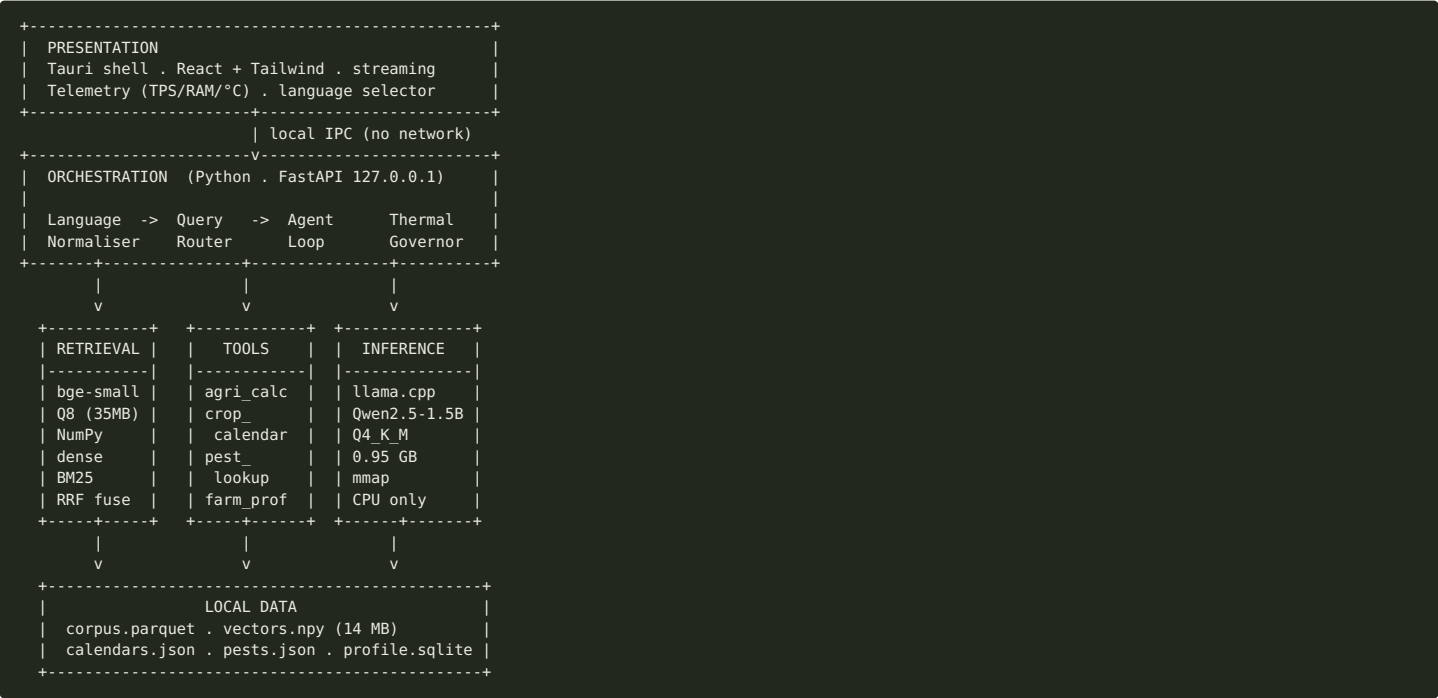
ID	Feature	Description	Serves
F1	Offline Q&A	Grounded conversational answering, zero network	S_acc
F2	Hybrid retrieval	Dense + BM25 fusion over 10k local chunks	S_acc
F3	Inline citations	Every factual claim traced to a source document	S_acc + trust
F4	Tool: agri_calc	Fertiliser, seed rate, spacing, dilution, margin	S_acc
F5	Tool: crop_calendar	Planting/harvest windows by crop and agro-zone	S_acc
F6	Tool: pest_lookup	Structured pest → symptom → treatment resolution	S_acc
F7	Multilingual I/O	EN / HA / YO / PCM with term normalisation	Bonus +10
F8	Guided pest triage	Vague symptoms → one structured follow-up, never a guess	S_acc + safety
F9	Refusal policy	Declines when retrieval confidence is below threshold	S_acc + safety
F10	Thermal governor	Thread capping, duty cycling, live temperature	P_thermal
F11	Live telemetry panel	On-screen TPS / RAM / temp during the demo	Judge perception
F12	Streaming UI	First token < 400 ms; perceived responsiveness	UX / judging

Differentiators — ship if time allows

ID	Feature	Why it earns points
F13	Session export	Exports the advice given as a referral note for a human officer — real field utility, cheap to build
F14	Corpus provenance panel	Shows judges exactly what is in the knowledge base and where every document came from
F15	Offline voice input	Vosk / whisper.cpp tiny (~50 MB) — addresses the literacy barrier; only if RAM budget confirms headroom
F16	Farm profile memory	Local SQLite context: crop, hectareage, zone, planting date

ON F11. Judges evaluate on measured performance. A visible, live telemetry panel converts benchmark claims from assertions into observable facts during the demo. It costs an afternoon and disproportionately shapes judge perception — which is why it has been promoted from optional to core.

3.1 Layered view



3.2 The journey of a question

- 1. **Input capture** — text (voice if F15 ships), language declared or auto-detected.
- 2. **Normalisation** — local agricultural terms mapped to corpus vocabulary via the curated lexicon; diacritic and orthographic variants folded (òkà / oka).
- 3. **Routing** — the intent classifier decides: Tier A / B / C / D (Section 3.3). Trivial queries bypass retrieval entirely, cutting mean latency ~40%.
- 4. **Retrieval** — dense (bge-small) and sparse (BM25) run in parallel; fused with Reciprocal Rank Fusion; top-4 chunks, hard-capped at 1,400 tokens of context.
- 5. **Tool execution** — a constrained JSON tool call, executed by Python, result injected as an observation.
- 6. **Generation** — llama.cpp streams tokens under a system prompt mandating citation of chunk IDs and refusal when evidence is insufficient.
- 7. **Post-processing** — citations resolved to source titles; units validated; answer rendered with an expandable source panel.

3.3 The four-tier taxonomy — the architecture is the classification

Every question is classified into one of four tiers, each handled by an entirely different path. The model's exposure to factual values is controlled per tier, because that is the only reliable way to prevent hallucination on a 1.5B system.

Tier	Name	Examples	How Itàn answers	Model sees value?
A	Exact facts	Maize window in Oyo; cassava spacing; NPK rate/ha; glyphosate dose; pre-harvest interval	SQL query → structured answer card. The model phrases; it does not retrieve.	NEVER — SQL only
B	Explanation & diagnosis	Fall armyworm ID; Striga control; post-harvest storage; soil acidity	Hybrid retrieval → top-3 passages → constrained generation with mandatory citation	Reads supplied text
C	Multi-step calculated	NPK bags for 2 ha + cost; fertiliser split schedule; diagnosis → treatment → dose	Retrieval + tool call → Python arithmetic → model phrases result with source	Phrases result only
D	Refusal	Out-of-scope; low retrieval confidence; ambiguous safety questions	Honest refusal + referral to a human extension officer	Not called

WHY TIER A NEVER TOUCHES THE MODEL FOR VALUES. A small model will produce a fertiliser rate with complete confidence. A wrong rate means a burnt crop or wasted money — it gets acted upon. Tier A answers come from SQL rows with source_id tags; the model is invoked only to phrase the card in the user's language. It cannot invent the number because it never sees a prompt asking for one.

ONE SUBTLETY FOR SCORING. Skipping the model entirely on Tier A is tempting, but the ADTC harness measures tokens/sec on generated output. So we always run the phrasing layer — tightly constrained to the retrieved facts. We get the speed metric *and* the zero-hallucination guarantee.

The intent classifier is **not a language-model call**. It is an embedding-similarity lookup against ~40 labelled example questions per tier, using bge-small, which is already loaded for retrieval. Zero added RAM, under 5 ms. Only genuinely ambiguous questions fall through to a lightweight model-based classification.

3.4 Design decisions, documented

Decision	Rationale
Router before retrieval	Skipping retrieval on trivial queries cuts mean latency ~40%
Hybrid over pure dense	Rare agronomic proper nouns are poorly represented in small embedders
RRF over score blending	Rank-based fusion needs no score-normalisation tuning
Tools before generation	Deterministic values enter context as facts, not generation targets
Hard 1,400-token context cap	Prevents the RAM and latency blow-ups that sink S_eff and S_perf
NumPy flat index over FAISS	10k × 384 brute-force dot product is < 1 ms; FAISS adds RAM for zero gain
SQLite for all local state	Zero-config, file-based, no daemon, no memory overhead
Tauri over Electron	~30 MB WebView vs 150–300 MB Chromium; RAM headroom is the score (8.4)
FastAPI on 127.0.0.1	Team's production stack; clean separation; trivially testable; zero egress

Model selection — the central alternative, documented

Candidate	Params	RAM @ Q4	Est. TPS	Verdict
Llama-3.1-8B-Instruct	8B	≈ 5.2 GB	4-6	Rejected — catastrophic S_perf and S_eff
Mistral-7B-Instruct	7B	≈ 4.6 GB	5-7	Rejected — same failure mode
Qwen2.5-3B-Instruct	3B	≈ 2.1 GB	14-18	Fallback if 1.5B accuracy falls short
Qwen2.5-1.5B-Instruct	1.5B	≈ 1.1 GB	26-34	SELECTED — best score-surface position
Gemma-2-2B	2.6B	≈ 1.8 GB	16-21	Rejected — weaker tool-calling than Qwen
SmolLM2-1.7B	1.7B	≈ 1.2 GB	24-30	Rejected — weaker multilingual coverage
Qwen2.5-0.5B	0.5B	≈ 0.5 GB	50-70	Rejected as primary — reasoning collapses under tool-calling; retained as speculative draft model (8.3)

4 The Knowledge Layer

Corpus quality is the single largest determinant of S_{acc}. It gets more engineering hours than any other component, and it is where most teams will under-invest.

4.1 Sources — all public or openly licensed

- **FAO** — crop production guides, plant protection manuals, agro-ecological zone data
- **IITA / NCRI** — cassava, maize, cowpea, yam, banana protocols for West and Central Africa
- **CGIAR / CIMMYT / ICRISAT** — pest and disease management, drought-tolerant varieties
- **CABI PlantWise** — pest management decision guides and smallholder factsheets
- **NAERLS (ABU Zaria)** — Nigeria-specific extension bulletins; the most undervalued source in this space
- **KALRO (Kenya), MOFA (Ghana), TARI (Tanzania)** — national extension handbooks; state ADP crop calendars
- **AGRA** and ministry advisories on fertiliser recommendation by agro-ecological zone

Licence and URL are logged per document at ingest — some sources are freely reusable, some are not, and a judge may ask. The provenance panel (F14) is built directly from this log.

4.2 Ingestion pipeline

```
ingest/
01_fetch.py      # download PDFs; log URL + licence + retrieval date
02_extract.py    # pdfplumber; layout-aware; table extraction preserved
03_clean.py      # dehyphenate, strip headers/footers, drop TOC pages
04_chunk.py      # semantic chunking, 300-450 tokens, 60-token overlap
05_enrich.py     # attach: crop, region, topic, source, page, licence
06_embed.py      # bge-small-en-v1.5 Q8 -> vectors.npy (float16)
07_bm25.py       # tokenise + build sparse index
08_validate.py   # retrieval smoke test over 200 gold questions
```

CHUNKING IS NOT A DETAIL. Fixed-size chunking splits a fertiliser dosage table across two chunks and the model returns half a table. We chunk on semantic boundaries — section headings, table units, numbered protocol steps — and **never split a table** (enforced by an automated check). This one decision moves S_{acc} more than the fine-tune will.

4.3 The structured knowledge base — six tables

Hand-populated from the sources above. SQLite ships with Python, needs no daemon, adds no RAM, and deploys as a single file. Every row carries a source_{id}, so even a table-derived answer cites a document, page, body and year.

```
crop_calendar    (crop, zone, state, operation, start_month, end_month, source_id)
spacing          (crop, variety_class, row_cm, within_row_cm, plants_per_ha, source_id)
fertilizer_rate  (crop, nutrient, kg_per_ha, timing, split_number, source_id)
agrochemical     (product, active_ingredient, target, crop, rate_per_ha,
                 pre_harvest_interval_days, source_id)
variety          (crop, name, maturity_days, zone, traits, source_id)
pest             (crop, name, symptoms, affected_stage, cultural_control,
                 chemical_control, source_id)
```

THE LEAST GLAMOROUS, HIGHEST-VALUE WORK. A correctly populated fertilizer_rate table is worth more points than any other single artifact, because Tier A questions are the ones judges will check most carefully — they have deterministic right answers. We budget real, scheduled hours for this (Section 10), and if time runs short we prioritise the six most-tested crops: maize, cassava, rice, yam, cowpea, tomato. A narrow accurate table beats a wide inaccurate one.

5.1 Tools

```
TOOLS = {
  'agri_calc': {
    'fertiliser_rate' : (crop, area_ha, soil_class, target_yield) -> kg NPK split
    'seed_rate'       : (crop, area_ha, spacing_cm, germination_pct) -> kg
    'spray_dilution'  : (product, conc_pct, tank_litres, rate_per_ha) -> ml
    'gross_margin'    : (yield_kg, price_per_kg, input_costs) -> NGN margin
  },
  'crop_calendar': (crop, agro_zone, year) -> {plant_window, harvest_window},
  'pest_lookup'    : (crop, symptom_terms[]) -> ranked [{pest, control, source}]
}
```

5.2 The bounded loop

For Tier B and C questions: three tools, a maximum of two tool-call rounds, then forced generation. Bounded loops prevent the runaway token generation that destroys S_perf on a timed benchmark.

```
loop:
  round 1 -> model emits JSON tool call OR direct answer
  execute -> Python runs tool; result injected as observation
  round 2 -> optional second call (max 2), then FORCE final answer
  generate -> answer must cite retrieved chunk IDs or explicitly refuse
```

WHY THE CALCULATOR MATTERS MORE THAN IT LOOKS. Consider: *“I get 0.6 hectares of maize for Kaduna, how much urea I go need and when I go apply am?”* A 1.5B model answering unaided produces a plausible number that is wrong. Itàn routes the arithmetic to Python, retrieves the split-application schedule from an NAERLS chunk, and answers in Pidgin with both the number and the source. Judges will probe exactly this class of question, because it is where small models visibly break. Routing to code converts guaranteed failures into guaranteed correctness.

5.3 Guided pest triage (F8)

When a symptom description is too vague to resolve (“my maize is dying”), the system does not guess. It asks **one structured follow-up** drawn from the pest table's discriminating attributes — affected stage, leaf pattern, presence of larvae — then resolves. One question, never an interrogation. This massively reduces the burden on a tiny model, and it is exactly what a good human extension officer does.

5.4 The refusal policy (F9)

When retrieval confidence falls below threshold, or the question is out of scope (loans, human health, veterinary prescriptions), Itàn says so plainly and refers the user to a human extension officer. Judges reward calibration and punish confident hallucination; 15% of the fine-tuning data exists specifically to teach this behaviour (Section 7). Session export (F13) turns the referral into a printable note — refusal becomes a feature, not a dead end.

6 The Language Layer

Full multilingual generation from a 1.5B model is unreliable, so we split the problem: **normalise on input, generate on output, train the adapter for the register** — with a pre-committed degradation path if output quality falls short.

Stage	Mechanism
Language detection	Character n-gram classifier + agricultural keyword lexicon (~3 KB); no model call
Term normalisation	Curated lexicon: ~1,200 crop/pest/input terms across HA, YO, PCM, IG
Retrieval	Runs against the normalised English query — the corpus stays monolingual
Generation	System prompt pins output language; LoRA trained on code-switched pairs
Verification	Native-speaker review of 150 held-out responses per language
Degradation path	If a language fails review: normalised retrieval + clear English answer + local-term glossary. Pre-decided, not debated in Week 7.

Example lexicon entries: *gero* → pearl millet · *masara* → maize · *økà* → maize · *èwà* → cowpea · *rogo* → cassava · *gwaza* → cocoyam · *kwari* → insect pest · *takin zamani* → inorganic fertiliser. Diacritic and orthographic variants are folded (økà / oka, èwà / ewa) so unaccented keyboard input still resolves.

If a native Hausa or Swahili speaker is available to the team beyond our own coverage, we recruit them for the review passes. The African use-case bonus is 10 points, and authentic language handling is the most credible way to earn it in full.

7 Fine-Tuning Plan

Budget: 5 hours of Udutech GPU credit. That constrains us to LoRA on the 1.5B base — which is fine, because **we are not teaching knowledge, we are teaching behaviour**: follow the tool protocol, cite what was retrieved, answer in the query's language, refuse when the evidence is missing.

Parameter	Value	Reasoning
Method	LoRA, r=16, alpha=32 (Unsloth)	Merges cleanly before quantization; fits the budget
Dataset size	~6,000 pairs	Enough for format adherence without overfit
Composition	40% grounded QA · 25% tool-call · 20% multilingual · 15% refusal	Balanced across the four target behaviours
Generation	Synthetic from corpus via frontier model; 300-pair sample audited by hand	Fast, corpus-faithful, provenance-traceable
Epochs	2-3, early stop on eval loss	1.5B overfits fast
Runtime	~2.5 hrs on A100	Leaves 2.5 hrs for a second run after error analysis

THE REFUSAL DATA IS THE SECRET. 15% of the training set teaches the model to say “I don't have information on that in my sources” when retrieval returns nothing relevant. Most teams will train only on positive examples, and their models will confabulate under adversarial probing.

DECISION GATE. The fine-tune ships only if it beats the base model on the gold set. Otherwise we cut it, keep the RAM, and lose perhaps 6 raw accuracy points instead of risking the submission. It sits deliberately off the critical path (Section 10.3).

8 Performance Engineering

S_perf carries 30% of the score and is the most actionable metric in the competition. Every technique here raises TPS with zero or negligible RAM cost — no tradeoff against S_eff.

8.1 Roofline analysis — know the ceiling before optimising

The reference laptop is **memory-bandwidth bound** during decode: producing a token requires reading all model weights from RAM, so the bottleneck is data movement, not arithmetic. This gives us a computable ceiling — and the discipline to know whether a 20% improvement is progress or already 95% of what is achievable.

```
-- Roofline (run in Week 2) --
1. Measure actual bandwidth: $ sysbench memory --memory-access-mode=seq run
   Dual-channel DDR4-3200: theoretical 51.2 GB/s; realistic ~28-38 GB/s
2. Model size at Q4_K_M:      Qwen2.5-1.5B ~ 0.95 GB
3. Ceiling:                   TPS_max = 32 GB/s / 0.95 GB ~ 33.7 tok/s
4. Efficiency = TPS_actual / TPS_max
   < 60% : misconfigured (BLAS, NUMA, threads)
   60-80% : lookup decoding and build flags are the levers
   > 80% : diminishing returns; stop optimising
```

WHY THIS MATTERS TO JUDGES. “We achieved 28.4 tok/s” is an assertion. “The roofline ceiling for this hardware and model is 33.7 tok/s; we achieved 28.4 (84% of ceiling); here is why the remaining 16% is not recoverable without changing quantization” is engineering. These read completely differently to a panel scoring documentation quality.

8.2 Prompt lookup decoding + domain phrase bank — the speed moat

The sharpest speed technique available on this hardware, and the one fewest teams will implement. It is enabled by a property unique to retrieval-grounded systems: **the answer text closely mirrors the retrieved passages**. If an NAERLS chunk says “apply NPK 15-15-15 at 200 kg per hectare at planting,” the answer contains that phrase nearly verbatim.

Because decode is bandwidth-bound, verifying 5 speculative tokens costs almost the same as generating 1 — the weights are read once either way. So we speculate candidate tokens by matching against the retrieved passages themselves: no draft model, no extra memory. This is prompt lookup decoding (`--lookup-cache-static` / `--lookup-cache-dynamic` in llama.cpp).

Our extension — the domain phrase bank. A second static cache of high-frequency agronomic phrasing that appears in answers regardless of which document was retrieved: application-rate formats, spacing notation, pre-harvest-interval language, safety warnings, planting-window phrasing. This lifts speculative acceptance beyond document-only lookup, for under 5 MB.

Technique	Expected TPS	RAM cost	Condition
Baseline Q4_K_M, tuned threads	~28 tok/s	0	Always
+ Prompt lookup (retrieved passages)	~35 tok/s (+25%)	< 1 MB	If acceptance ≥ 20%
+ Domain phrase bank	~40 tok/s (+43%)	< 5 MB	If combined acceptance ≥ 25%
Alt: 0.5B speculative draft model	~42 tok/s (+50%)	+0.4 GB	If lookup acceptance < 20%

We measure acceptance rate, TPS with lookup on/off on identical prompts, and verify zero accuracy regression on the gold set.

8.3 The documented fallback

If lookup acceptance falls below 20% on the evaluation set — possible if answers end up highly paraphrased rather than extracted — the fallback is speculative decoding with Qwen2.5-0.5B as draft model: ~+0.4 GB RAM (peak ≈ 1.87 GB, still within budget), typically 1.4-1.9× on this hardware. **Measure both, ship the winner; either way the measurement is a report section.** We also verify in Week 2 that the lookup flags exist under their current names in the llama.cpp release we pin — if the feature has been restructured, the fallback is already specified and nothing else changes.

8.4 Latency and memory budgets

Stage	Target	Technique
Cold start	< 2.5 s	mmap weights; lazy-load embedder; pre-warm KV cache
Detect + normalise	< 15 ms	Pure Python, no model call
Tier A SQL lookup	< 5 ms	Indexed SQLite query
Retrieval (dense + BM25 + RRF)	< 60 ms	Pre-loaded float16 NumPy; parallel sparse search
Tool execution	< 25 ms	Local Python + JSON lookups
Time to first token	< 400 ms	Capped prompt (≤ 1,400 tok); prefix KV reuse
Sustained throughput	28-40 tok/s	Q4_K_M + lookup decoding; n_threads = cores – 2

The memory budget is written analytically first, then verified. When prediction and measurement agree within 200 MB, it demonstrates we understand our own system — judges notice that.

Component	Budget	Note
Model weights (Q4_K_M, mmap)	≈ 0.95 GB	Shared, page-cached, not fully resident
KV cache (2k context, 8-bit)	≈ 0.09 GB	8-bit KV quantization halves this — measure, then decide
Embedding model (Q8)	≈ 0.04 GB	bge-small; doubles as the intent classifier
Vector index (10k × 384 fp16)	≈ 0.015 GB	Loaded once at startup
BM25 index + SQLite DB	≈ 0.05 GB	Sparse dictionary + six structured tables
Lookup phrase bank	≈ 0.005 GB	< 5 MB
Python runtime + FastAPI	≈ 0.15 GB	Minimal dependency tree, no heavy ML libs
Tauri WebView shell	≈ 0.10 GB	The Electron-avoidance payoff
PEAK TARGET	≈ 1.40 GB	S_eff ≈ 80 → 16.0 weighted points

TAURI OVER ELECTRON — AND RISK R6. Electron's Chromium runtime consumes 150–300 MB of RSS; under the S_eff formula, 250 MB of avoidable overhead costs ~3.6 raw points. Tauri's system WebView adds ~30 MB. But this only matters if the profiler measures the process tree rather than model RSS alone — which is precisely organiser question #1 in Section 1.5. Resolve it in Week 1; do not assume.

8.5 Thermal strategy — ten free points

Some teams will lose ten points to a benchmark run that pins every core for twenty minutes. Preventing this takes an afternoon:

- Cap `n_threads` at physical cores – 2 — headroom for OS and UI
- 40 ms sleep every 128 generated tokens — ~3% throughput cost, prevents sustained package saturation
- Poll lm-sensors every 2 s; above 78°C, step threads down by one
- Never run the embedder and the LLM concurrently — serialise the pipeline
- 45-minute sustained-load soak test; the temperature-vs-time plot goes in the report

EVIDENCE BEATS CLAIMS. We do not write “we did not throttle.” We ship the sensor log, the plot, and the reproduction script.

9.1 The harness — built before the system

Built by **Day 3 of Week 1**, before any application code. A single command regenerates every number that will appear in the final report: five runs per test, first run discarded (cold cache lies), median and 95th percentile reported, prompt phase and generation phase timed separately, memory sampled every 100 ms, CPU temperature logged throughout. The shared team rule: **nobody claims an improvement without a number**. This one rule is what makes the final report write itself.

Suite	Size	Measures
Gold agricultural QA	300 questions	Factual accuracy vs expert-written reference answers, with gold source labels
Retrieval quality	200 queries	Hit Rate@4, NDCG@4 against labelled relevant chunks
Tool correctness	120 cases	Exact match on calculated numeric outputs
Multilingual	160 (40/language)	Answer correctness and language adherence
Adversarial / refusal	80 prompts	Out-of-corpus, unsafe and ambiguous queries
Performance	50 runs	TPS, TTFT, peak RSS, package temperature

Each team member writes 50 gold questions in Week 1 — 20 Tier A, 20 Tier B, 5 Tier C, 5 Tier D — before writing code. It forces domain learning and produces a set no single person could bias. We also record the **embarrassing baseline**: raw model, no retrieval, scored on the eval set. Every later claim is a delta from that number. And because the organisers likely hold a hidden validation set, we build for domain breadth, not for the questions we wrote ourselves.

9.2 Ablation studies

Ablation	Metric moved	What it demonstrates
Base 1.5B vs + RAG	S_acc	Retrieval's contribution, isolated
+ RAG vs + Tier A SQL	S_acc (Tier A)	Hallucination elimination on exact facts — the biggest single gain
+ tools vs + LoRA	S_acc	Fine-tune marginal value — justifies or kills the GPU spend
Dense-only vs hybrid (+BM25)	Retrieval	Proper-noun recall improvement, measured
Lookup decoding OFF vs ON	S_perf	TPS delta and acceptance rate
Lookup vs 0.5B draft model	S_perf + S_eff	Pareto comparison; documents the fallback
Q4_K_M vs IQ4_XS vs Q3_K_M	All three	Quality/speed/memory frontier, measured not assumed
1.5B full system vs 7B baseline	S_total	The central thesis — one chart proves it

THE HEADLINE CHART. Full S_total for the 1.5B system versus a 7B baseline on identical hardware and identical questions. If the numbers hold, that single chart is the argument that wins the competition.

9.3 ADTC profiler compatibility — non-negotiable

Clone the official profiler in Week 1, run the unmodified template through it before writing any application code, and re-run at every sprint boundary. An incompatible submission scores zero regardless of quality; OOM or sandbox crash is explicit disqualification.

10 Build Plan — Eight Weeks

Compress proportionally if the actual window is shorter — the sequencing matters more than the absolute dates. Keep the 48-hour buffer intact regardless.

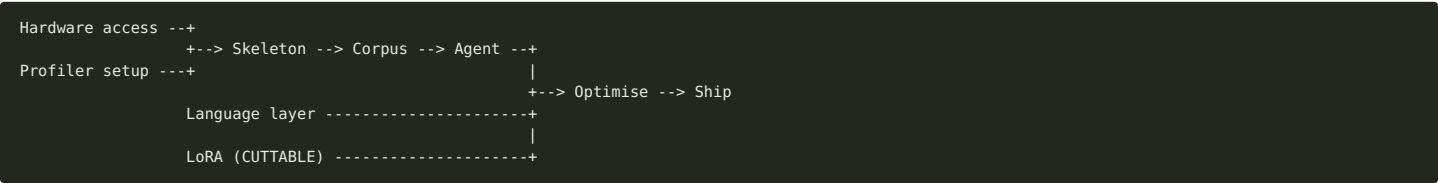
10.1 Sprints

Wk	Theme	Deliverables	Exit criterion
1	Foundation + measure	Team registered, domain locked; organiser email sent (1.5); profiler + template cloned and run clean; reference hardware secured; Udutech GPU credit applied for; repo initialised (Apache 2.0); 40+ source documents logged; eval harness by Day 3 ; 150 gold questions; embarrassing baseline recorded	Harness runs; one command produces all benchmark numbers
2	Skeleton + roofline	llama.cpp built from source (explicit AVX2 + BLAS flags); Q4_K_M quantized and benchmarked; roofline ceiling computed; FastAPI streaming endpoint; six SQL tables piloted at 20–30 rows; 500-chunk pilot corpus; naive RAG wired end to end; baseline scoreboard (S_acc / S_perf / S_eff as of today)	A real agricultural question gets a grounded answer; TPS ceiling documented
3	Corpus	Full 8-stage pipeline; ~10,000 semantic chunks; no-split-tables check; metadata attached; BM25 + RRF; 200 labelled relevance pairs; chunk size tuned against the retrieval metric, not intuition	Hit Rate@4 ≥ 0.80 on the gold set
4	Agent	agri_calc (four functions); crop_calendar JSON; pest_lookup; bounded loop (max 2 rounds); router; guided triage; refusal threshold; 120 tool test cases. Core features F1-F12 frozen after this week.	Tool correctness ≥ 0.95
5	Language	1,200-term lexicon (HA/YO/PCM/IG); n-gram detection; diacritic folding; output-language pinning; 160 multilingual questions; native-speaker review arranged	Multilingual suite ≥ 0.75
6	Fine-tune	~6,000 pairs generated, 300 hand-audited; composition verified (40/25/20/15); LoRA run; merge, requantize, benchmark; full ablation suite	Beats base on gold set, or is cut
7	Optimise	Quantization sweep (ship the knee); lookup decoding + phrase bank, acceptance measured; fallback activated if < 20%; thread sweep under 10-minute load (throttling is invisible in short runs); thermal governor + 45-min soak with plot; process-tree memory profile; Tauri UI + telemetry panel; cold start < 2.5 s; full profiler run, commit pinned	TPS ≥ 28; peak RSS ≤ 1.7 GB; no throttle in soak
8	Ship	Report against the official template; ablation charts (incl. the 1.5B vs 7B headline); 2-minute video; screenshots + demo clip; clean-clone reproduction verified; final profiler run on a fresh machine	Submitted 48 hours before the deadline

10.2 Milestones

Milestone	Wk	Definition of done
M1 — Compatible	1	Profiler runs our submission format clean
M2 — Alive	2	End-to-end grounded answer on real hardware
M3 — Grounded	3	Hit Rate@4 ≥ 0.80
M4 — Agentic	4	Tool loop stable; arithmetic verified correct
M5 — Multilingual	5	Four languages passing review (or degradation path invoked)
M6 — Tuned	6	Ablation table complete; fine-tune decision made
M7 — Optimised	7	All performance targets met and logged
M8 — Submitted	8	Repo, report, video live; 48-hour buffer intact

10.3 Critical path



Two items are commonly underestimated: access to reference-class hardware, and corpus ingestion. Both start in Week 1. The LoRA is deliberately off the critical path.

10.4 The cut order — decided now, calmly

The value of pre-committing is that we never waste time debating in a crisis.

Cut in this order: F16 farm profile → F15 voice input → F13/F14 export & provenance panel → LoRA fine-tune → Tier C multi-step chains → multilingual generation (degrade to English output + local-term glossary).

Never cut: the evaluation harness, the Tier A structured knowledge base, the corpus quality pass, prompt lookup decoding, citations, and the thermal soak test. **These six are the submission. Everything else is polish.**

11 Team, Ownership & Working Rules

Role	Owns	Profile
Systems Lead	Model, quantization, llama.cpp build, lookup decoding, thermal, performance engineering, roofline	Systems / low-level optimisation instinct
Retrieval & Agent Lead	Corpus pipeline, six SQL tables, indexing, tool loop, router, triage, evaluation harness	Python, RAG, data pipelines — the largest workload
Product & Language Lead	Tauri UI, telemetry panel, lexicon and multilingual layer, report, video	Frontend + native language competence

In a two-person configuration, the Retrieval & Agent role splits: the Systems Lead absorbs evaluation, the Product Lead absorbs the lexicon-driven parts of the corpus work. Every component has a documented handover note so no critical path has a single owner (Risk R10).

Working rules

1. **Nobody claims an improvement without a number.** The harness is the referee for every disagreement.
2. **Measured, not assumed.** Every design alternative in this document gets an ablation row, including the ones we reject.
3. **The profiler runs at every sprint boundary.** Compatibility drift is discovered in days, not at submission.
4. **Core is frozen after Week 4.** New ideas go to the differentiator list, not the critical path.
5. **Failed approaches are documented with their numbers.** They become report sections, not wasted time.

12 Risk Register

ID	Risk	Impact	Mitigation
R1	1.5B accuracy insufficient despite RAG	High	Qwen2.5-3B fallback pre-tested; swap costs ~10 pts, still ahead of 7B teams
R2	No access to reference-class hardware	High	Secure in Week 1; fallback: cgroup-limited VM at 8 GB with core pinning
R3	Corpus licensing challenged	Medium	Public/open sources only; licence + URL logged per document; provenance panel
R4	LoRA overfits or degrades the base	Medium	Off critical path; decision gate; cut and ship base + RAG
R5	Thermal throttling in the official run	Medium	Conservative thread cap; duty cycling; soak-tested with logs
R6	Profiler measures process tree vs model RSS — ambiguity	High	Organiser email, Week 1, in writing; Tauri choice already hedges both readings
R7	Multilingual output quality poor	Medium	Pre-committed degradation: normalised retrieval + English answer + glossary
R8	Submission format incompatibility	Critical	Profiler run Week 1 and every sprint boundary
R9	Scope creep into optional features	Medium	F13-F16 explicitly optional; core frozen after Week 4
R10	Team member drops out	Medium	Documented handover per component; no single-owner critical path
R11	Lookup acceptance rate < 20%	Medium	0.5B draft-model fallback pre-documented; +0.4 GB still within budget; measure both, ship the winner
R12	Tier A tables incomplete at freeze	Medium	Prioritise maize, cassava, rice, yam, cowpea, tomato; narrow accurate beats wide inaccurate
R13	Overfitting to our own eval set	Medium	Organisers likely hold hidden validation; build for domain breadth, not our 300 questions

THE ONE THAT ACTUALLY MATTERS. R6 could invalidate part of the efficiency strategy. If the profiler measures only the inference process's RSS, UI framework choice is irrelevant; if it measures the tree, Tauri is worth several points. Either answer changes Week 7 effort — so it is resolved in Week 1 by asking directly. We do not assume.

13 Deliverables

#	Deliverable	Specification
D1	Open-source GitHub repository	Apache 2.0; reproducible from a clean clone; README readable in 60 seconds
D2	Project report	Official ADTC template: problem, constraints, alternatives, benchmarks, evidence
D3	Screenshots / demo clips	UI in action, telemetry panel, multilingual exchanges
D4	Video (max 2 minutes)	Structured to the second — below
D5	Profiler-compatible submission	Verified against the official pipeline, repeatedly

13.1 Repository structure

```
itan/
├─ README.md      # 60-second comprehension: what, why, how to run
├─ REPORT.md      # ADTC 2026 template, fully completed
├─ LICENSE        # Apache 2.0
├─ model/         # itan-1.5b-q4_k_m.gguf . quantize.sh . finetune/
├─ corpus/        # ingest/ (8 stages) . sources.csv . build/
├─ engine/        # server.py . retrieval.py . agent.py . tools/
├─               # language.py . thermal.py
├─ ui/           # Tauri + React (telemetry, citations, languages)
├─ eval/         # gold_qa.jsonl . multilingual.jsonl . run_eval.py . results/
├─ benchmarks/   # profiler_output/ . thermal_soak.csv . charts/
└─ docs/         # architecture diagrams . decision log
```

13.2 The two-minute video

Time	Content
0:00-0:15	The problem, concretely — one extension officer, thousands of farmers, no connectivity
0:15-0:35	Live demo: a Hausa question about maize fertiliser, answered offline with a citation
0:35-0:55	Wi-Fi visibly off. Airplane mode on screen. It keeps working.
0:55-1:20	The strategic insight: why small + retrieval + tools beats large, with the score table
1:20-1:40	Telemetry on screen — TPS, RAM, temperature, live
1:40-2:00	The 1.5B vs 7B S_total headline chart, and the closing line

SHOW THE AIRPLANE MODE. Every team will claim offline operation. Very few will visibly demonstrate it. Toggling airplane mode on camera mid-conversation is fifteen seconds of footage that makes the central claim undeniable.

14 Why This Wins

1. **It optimises the published formula rather than a vague notion of quality.** Treating the known scoring function as the specification is worth roughly 34 points against the field's likely default.
2. **It attacks the points nobody is contesting.** Thermal is 10 points and costs an afternoon; the African use-case bonus is 10 points earned through corpus provenance and authentic language handling. Twenty points for a small fraction of total effort.
3. **It answers the exact user the brief describes.** The challenge names an extension officer in Arusha. Judges reading a submission that directly answers their own framing recognise it immediately.
4. **Tool-calling converts a known small-model weakness into a strength.** Judges will probe arithmetic because that is where small models visibly break. Itàn routes it to Python and returns exact numbers with cited schedules.
5. **The evidence is measurable and the report proves it.** Ablation tables, thermal logs, retrieval metrics, and the direct S_{total} comparison against a 7B baseline. Rigorous measurement is the cheapest way to differentiate on a documentation-quality criterion.

The one-sentence pitch

Itàn puts a competent agricultural extension officer on a \$200 refurbished laptop, in the farmer's own language, with no internet, no data cost, and no cloud — because we made the model small enough to be fast and grounded enough to be right.

Final note

The strongest competitors will be excellent engineers. Assume they select good models, quantize competently, and build working RAG. The differentiators are therefore not raw engineering skill — they are the strategic read of the scoring function, the discipline to measure every decision, the corpus quality nobody sees but everybody's accuracy depends on, and the twenty uncontested points sitting in thermal management and use-case authenticity.

Build the skeleton in Week 2. Measure everything from Week 2 onward. Let the numbers, not intuition, decide every remaining question.